BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
MUNSHI NAGAR, ANDHERI (WEST), MUMBAI – 400 058.
(Autonomous College Affiliated to University of Mumbai)
MASTER OF COMPUTER APPLICATIONS
ACADEMIC YEAR: 2017-18

Course: Operating System Class: FYMCA Sem: II Course Code: MCA21

Q1. Classify different types of OS
Answer: List the different types=01 marks
Functioning of each type carries 1 mark(any 4)

1. Simple Batch System
2. Multiprogramming System
3. Multiprocessor System
4. Desktop System
5. Distributed Operating System
6. Clustered System
7. Real time Operating System
8. Handheld System

Batch OS:

- The user has to submit a job (written on cards or tape) to a computer operator.
- Then computer operator places a batch of several jobs on an input device.
- Jobs are batched together by type of languages and requirement.
- Then a special program, the monitor, manages the execution of each program in the batch
- No direct interaction between user and the computer

Multiprogramming System

- In this the operating system picks up and begins to execute one of the jobs from memory.
- Once this job needs an I/O operation operating system switches to another job (CPU and OS always busy)
  - If several jobs are ready to run at the same time, then the system chooses which one to run through the process of **CPU Scheduling.**

Time-Sharing Systems/Multitasking System

- **Time-Sharing Systems** are very similar to Multiprogramming batch systems. In fact time sharing systems are an extension of multiprogramming systems.
- In time sharing systems the prime focus is on minimizing the response time, while in multiprogramming the prime focus is to maximize the CPU usage.

Real Time OS

- The Real-Time Operating system which guarantees the maximum time for critical operations and complete them on time are referred to as **Hard Real-Time Operating Systems.**
- While the real-time operating systems that can only guarantee a maximum of the time, i.e. the critical task will get priority over other tasks, but no assurity of completeing it in a defined time. These systems are referred to as **Soft Real-Time Operating Systems.**

Q1. Illustrate different types of system calls

System Call

(Definition=01 mark)

When a program in user mode requires access to RAM or a hardware resource, it must ask the kernel to provide access to that resource. This is done via something called a **system call**.

When a program makes a system call, the mode is switched from user mode to kernel mode. This is called a **context switch**.

Then the kernel provides the resource which the program requested. After that, another context switch happens which results in change of mode from kernel mode back to user mode.

Types: 01 mark for each type explanation

Generally, system calls are made by the user level programs in the following situations:

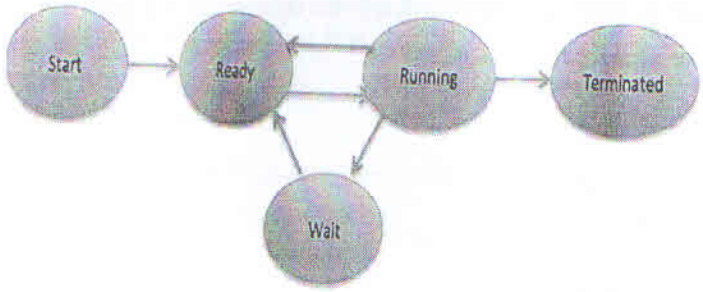
- File Management: Creating, opening, closing and deleting files in the file system.
- Process control: Creating and managing new processes (load, execute, end, and abort).
- Communication: Creating a connection in the network, sending and receiving packets.
- Device Management: Requesting access to a hardware device, like a mouse or a printer ,release device, read, write, get device attributes etc

Q2. Demonstrate process state diagram

Process

A process is basically a program in execution. The execution of a process must progress in a Sequential fashion

Diagram=02 mark



Working= 03 marks

Start

This is the initial state when a process is first started/created.

Ready

The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after **Start** state or while running it by but interrupted by the scheduler to assign CPU to some other process

Running

Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

Waiting

Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.

Terminated or Exit

Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

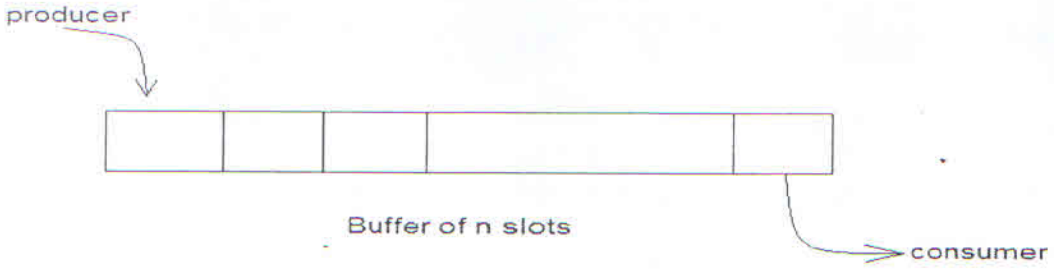
Q3. There is a buffer of 5 slots and each slot is capable of storing one unit of data. There are two processes running, namely, producer and consumer, which are operating on the buffer. producer produces an article and consumer consumes it. How does this problem can be solved using semaphores?

semaphore is a variable or abstract data type used to control access to a common resource by multiple processes in a concurrent system such as a multitasking operating system which can be accessed by to atomic operation wait() and signal()

Definition=01 mark

Problem statement with diagram=01 mark

There is a buffer of n slots and each slot is capable of storing one unit of data. There are two processes running, namely, **producer** and **consumer**, which are operating on the buffer.



A producer tries to insert data into an empty slot of the buffer. A consumer tries to remove data from a filled slot in the buffer

Solution (03 marks):

One solution of this problem is to use semaphores. The semaphores which will be used here are:

- **m**, a binary semaphore which is used to acquire and release the lock.
- **empty**, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty.
- **full**, a counting semaphore whose initial value is 0.

At any instant, the current value of empty represents the number of empty slots in the buffer and full represents the number of occupied slots in the buffer.

Producer Operation:

The pseudocode of the producer function looks like this:

```
do {
   wait(empty);   // wait until empty>0 and then decrement 'empty'
   wait(mutex);   // acquire lock
   /* perform the insert operation in a slot */
   signal(mutex); // release lock
   signal(full);  // increment 'full'
} while(TRUE)
```

- Looking at the above code for a producer, we can see that a producer first waits until there is atleast one empty slot.
- Then it decrements the **empty** semaphore because, there will now be one less empty slot, since the producer is going to insert data in one of those slots.
- Then, it acquires lock on the buffer, so that the consumer cannot access the buffer until producer completes its operation.

- After performing the insert operation, the lock is released and the value of **full** is incremented because the producer has just filled a slot in the buffer.

Consumer Operation:

The pseudocode of the consumer function looks like this:

```
do {
 wait(full); // wait until full>0 and then decrement 'full'
 wait(mutex);  // acquire the lock
 /* perform the remove operation    in a slot */
 signal(mutex); // release the lock
 signal(empty); // increment 'empty'
} while(TRUE);
```

- The consumer waits until there is atleast one full slot in the buffer.
- Then it decrements the **full** semaphore because the number of occupied slots will be decreased by one, after the consumer completes its operation.
- After that, the consumer acquires lock on the buffer.
- Following that, the consumer completes the removal operation so that the data from one of the full slots is removed.
- Then, the consumer releases the lock.
- Finally, the **empty** semaphore is incremented by 1, because the consumer has just removed data from an occupied slot, thus making it empty.

Q.6 Given string, 3 frames.

	1	2	3	2	1	5	2	1	6	2	5	6	3	1	3	6	1	2	4
	1	1	1	1	1	1	1	1	6	6	6	6	6	6	6	6	6	2	2
		2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	4
			3	3	3	5	5	5	5	5	5	5	3	3	3	3	3	3	3
	*	*	*			*			*				*	*				*	*

Total no of page faults = 09.

Q.5 Banker Algo

	Max A B C	Allocation A B C	Available A B C
P_0	0 0 1	0 0 1	
P_1	1 7 5	1 0 0	
P_2	2 3 5	1 3 5	
P_3	0 6 5	0 6 3	
Total		2 9 9	1 5 2

Need = Max − Allocation (01 marks)

Need Matrix =

	A	B	C
P_0	0	0	0
P_1	0	7	5
P_2	1	0	0
P_3	0	0	2

Safe sequence = $< P_0, P_2, P_1, P_3 >$ (04 marks)

Q3: Use RR scheduling with TQ = 3.

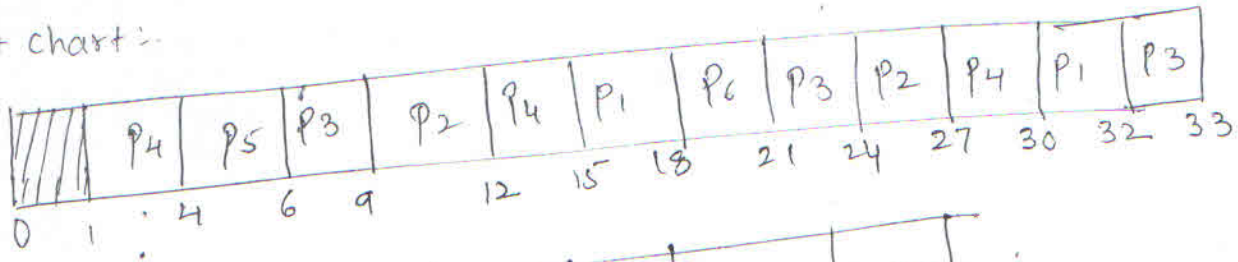
Process NO	AT	BT
1	5	5
2	4	6
3	3	7
4	1	9
5	2	2
6	6	3

Ans: Gantt chart = 01 mark.

Avg waiting time Calculation = 02 marks.

Avg Turnaround time Calculation = 02 marks.

Gantt chart:-



Process NO	AT	BT	CT	TAT	WT
1	5	5	32	27	22
2	4	6	27	23	17
3	3	7	33	30	23
4	1	9	30	29	20
5	2	2	6	4	2
6	6	3	21	15	12

Avg WT = (22+17+23+20+2+12) / 6 = 16

Avg TAT = (27+23+30+29+4+15) / 6 = 21.33

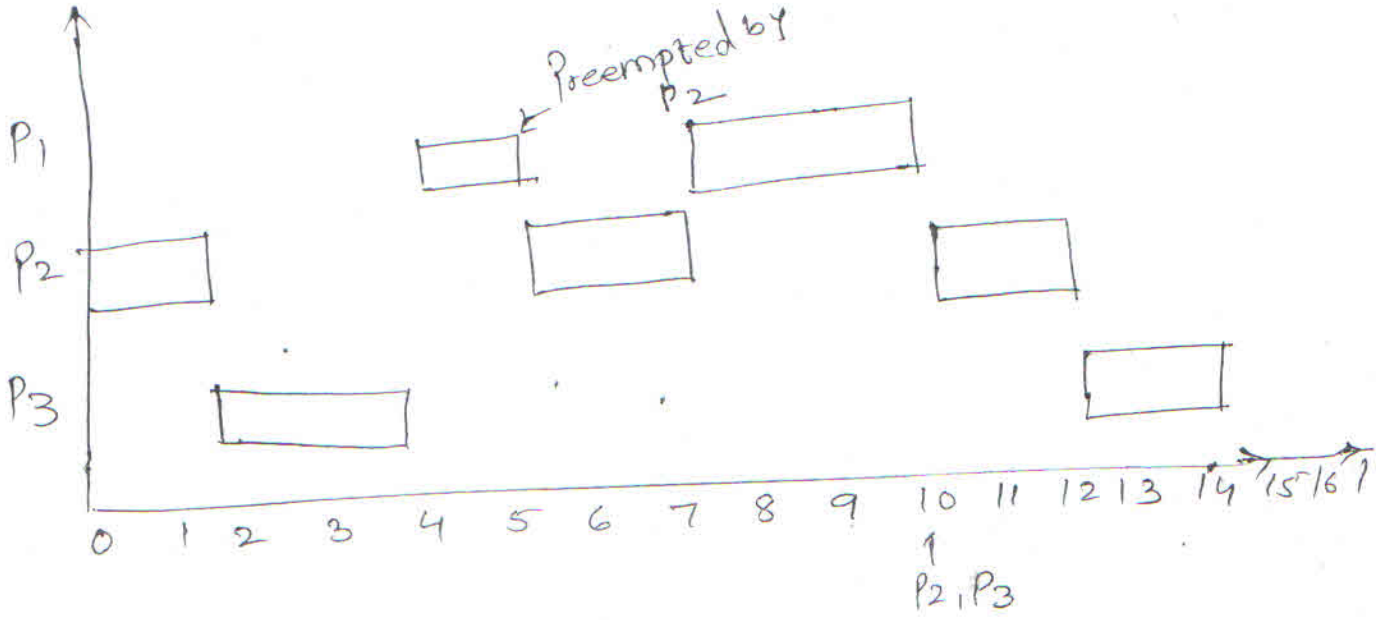
Q5:

PNO	Priority	Time Period	Service Time
1	3	20	3
2	1	5	2
3	2	10	2

Ans:- Gantt chart : 03 marks. [Diagramatic representation]

calculation of Utilization bound = 01 mark.

Conclusion = 01 mark



[CPU utilization] = $U = \frac{3}{20} + \frac{2}{5} + \frac{2}{10} = 0.15 + 0.4 + 0.2 = 0.75$

$U_{bound} = n\left(2^{1/n} - 1\right) = 3\left(2^{1/3} - 1\right) = 77.97$

$U < U_{bound}$ (i.e. $75 < 77.97$)

∴ given set of tasks are schedulable.